



Transaction Scheduling

Content

Transaction Isolation

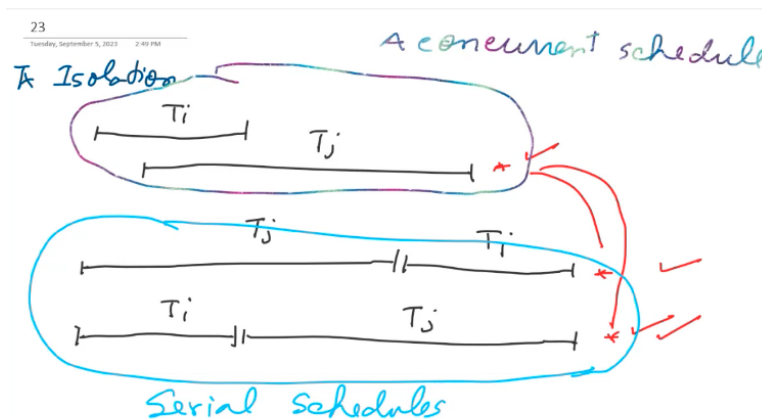
(about multitasking, concurrent execution)

(we can handle multiple transaction during same period of time, because the speed of i/o is much slower than speed of cpu, so we can handle many transaction that is basically i/o bound → use more i/o time than cpu)

- But the challenges is how to obtain correct result?

Let moved back to ACID properties: Isolation

- Isolation: Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.
 - Intermediate transaction results must be hidden from other concurrently executed transactions



- The last approach is most popular, but in practical the second is also acceptable because it just a fraction of second, we don't really know which is which.
- The second and last one called → serial schedule



We want concurrent schedule that give the same result as serial schedule → we call that a concurrent serializable schedule

What is Schedule?



Schedules

- **Schedule** – a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed
 - A schedule for a set of transactions must consist of all instructions of those transactions
 - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
 - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement

chronological order → order according to time.

A schedule is an arrangement of instructions execution. There are several transaction in a schedule. Each transaction comprises several instructions.

- Schedule is basically a sequence of instruction execution (execution arrangement)
- 1 schedule typically comprised of many several transaction → each transaction comprised many instruction.
- **Serial schedule** → instruction that belong in the same transaction is execute uninterrupt from the beginning to the end of transaction.
- **Concurrent schedule** → we may execute instruction from different transaction, sometime we executed from the first tx, and switch,....



In practice, we don't need serial schedule, we need concurrent schedule, however we need concurrent serializable schedule.

Serializable schedule (concurrent that gave the same result as serial schedule) $\neq$ serial schedule

Level of Consistency



Levels of Consistency in SQL-92

- **Serializable** — default
 - **Repeatable read** — only committed records to be read.
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
 - **Read committed** — only committed records can be read.
 - Successive reads of record may return different (but committed) values.
 - **Read uncommitted** — even uncommitted records may be read.
-
- we don't have to be serializable at all time → can change
 - you can adjust to be anything
 - In practice, many people not set their system to be serializable, and when ask why, they reply something like we don't want our transaction to be run in sequence. They misunderstood the term serializable and serial



if we choose serializable → it mean conflict serializable schedule (17.6 not 17.8) (most of the time)

2 samples transaction

According to the book, the book gave us 2 sample transaction

1. Transaction comprised only sql statement (2 update statement)

```

T1: read(A);
    A := A - 50;
    write(A);
    read(B);
    B := B + 50;
    write(B).

```

- T1: Money transfer transaction, transfer 50 dollars from A to B
 - read(), write() → update statement
- 2. Transaction comprised of both sql statement and application program statement (refer to program variable inside the source program)

```

T2: read(A);
    temp := A * 0.1;
    A := A - temp;
    write(A);
    read(B);
    B := B + temp;
    write(B).

```

- T2: Read A and reduce 10% from A using temp, write A, read B, add that temp to B and write B
 - how to implement → calculate temp first
 - read that row A → calculate the temp in the program, you read row B reduce temp from A, add temp to B and write back(can be done using embeded sql)
 - so in embeded sql → you select particular row, modify the value and write back to the row.

Example

we want concurrent schedule that give the same result as the serial one

Diagram 17.2

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.2 Schedule 1—a serial schedule in which T_1 is followed by T_2 .

- A serial schedule
- T_1 run until commit, and then follow by T_2

Diagram 17.3

T_1	T_2
	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	

Figure 17.3 Schedule 2—a serial schedule in which T_2 is followed by T_1 .

- A serial schedule
- T_2 run first, follow by T_1

Diagram 17.4

- Concurrent schedule that gave the same result as 17.2
 - you can move read(A), write(A) of T2 down and move read(B) and write(B) to T1 up

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code>	
	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code>
<code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code> <code>commit</code>	
	<code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code> <code>commit</code>

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

- concurrent schedule (not a serial one)
- this one is the concurrent schedule that give the same result at 17.2
 - but how do you know, like us we human can be able to tell, but how dbms know (this is our challenge)
- we know that this is same result and correct as 17.2
 - we considered the calculation to decide, but the problem is for t1 is okay, but for t2, t2 has calculation in source program, it beyond the scheduler, dbms can't access source code. (or else one day, all run in the cloud)

Diagram 17.5(another concurrent schedult, give wrong result)

- we refer 17.4 because it gave the same result as 17.2

T_1	T_2
$\text{read}(A)$ $A := A - 50$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$
$\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$B := B + \text{temp}$ $\text{write}(B)$ commit

Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

- a concurrent schedule (not a serial one)
- T_2 write first before $T_1 \rightarrow$ when T_1 write (the value is incorrect) $\rightarrow$ missequence
 - we should be able to detect this without calculation.
- T_2 should have read A which was produce by T_1
- not conflict serailizable $\rightarrow$ can't transform to serial $\rightarrow$ because write (A) of T_2 and write(A) of T_1 is actually conflict.

How do we know that it correct (we can't determine by comparing result, some can't access all the code)



We have to have algorithm, we have to have a standard on what is correct and what is not.

- the dbms know the read and write sequence, the dbms receive read andwrite sequence from application
 - from the read , write sequence $\rightarrow$ we'll be able to tell whether it correct or not(same result as serial schedule)
 - try to swap non conflict instruction

Diagram 17.6 (show only read and write)

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Figure 17.6 Schedule 3—showing only the read and write instructions.

- 17.4 with only read and write

Diagram 17.7

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

Figure 17.7 Schedule 5—schedule 3 after swapping of a pair of instructions.

- swap some instruction from 17.6
- still same result because they work on different data item
 - write(A) first, read(B) first doesn't matter

Diagram 17.8

- swap instruction of 17.6 to have a serial schedule

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

Figure 17.8 Schedule 6—a serial schedule that is equivalent to schedule 3.



Some Technical term

- Conflict equivalent
 - if a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.

🌱 Diagram 17.9 (not a conflict serializable schedule, gave wrong result)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.
- this gave incorrect result

🌱 Diagram 17.x (non conflict serializable that give correct result)

- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		
		write (Q)

- What serial schedule is above equivalent to?
 - Every view serializable schedule that is not conflict serializable has **blind writes**.
- this give correct result → same as serial ($T_{27}, 28, 29$)
 - not conflict serializable → can't swap write of T_{27} and T_{28}
 - if you run in sequence, the result of Q will be the one that write by T_{29}

Diagram 17.13 (non conflict that give correct result)

- The schedule below produces same outcome as the serial schedule $\langle T_1, T_5 \rangle$, yet is not conflict equivalent or view equivalent to it.

T_1	T_5
read (A) $A := A - 50$ write (A)	read (B) $B := B - 10$ write (B)
read (B) $B := B + 50$ write (B)	
	read (A) $A := A + 10$ write (A)

- Determining such equivalence requires analysis of operations other than read and write.
- produce same output, but not conflict equivalent to serial

Standard of correctness (concurrency control based on correctness concept)

Conflict Serializability

- so if a schedule S can be transform into schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are conflict equivalent
 - and if schedule conflict equivalent to a serial schedule → we say that a schedule S is conflict serializable.
 - conflict serializable is standard of correctness → if we said this schedule is conflict serializable → it's mean it correct

Conflict Serializability (Cont.)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

T_1	T_2	T_1	T_2
read (A)		read (A)	
write (A)		write (A)	
	read (A)		
	write (A)		
		read (B)	
read (B)		write (B)	
write (B)			read (A)
	read (B)		write (A)
	write (B)		read (B)
			write (B)

Schedule 3

Schedule 6



we can said that 17.6, 17.7, 17.8 are conflict equivalent schedules.

and because 17.8 is a serial schedule, 17.6 and 17.7 are conflict serializable schedule. (They are conflict equivalent to a serial schedule)

What are those non-conflicting instruction?

- we can swap read(A), write(A) of T_2 with read(B) write(B) of T_1 , simply because the two work on different data item
- instruction that work on different data item not conflict, naturally

in the case, that the instruction from 2 transactions works on the same data item.



Conflicting Instructions

- Instructions I_i and I_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both I_i and I_j , and at least one of these instructions wrote Q .
 - $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j don't conflict.
 - $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They conflict.
 - $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They conflict.
 - $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They conflict.
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them.
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

- if T_i, T_j both read Q not matter $\rightarrow$ before, after don't matter
- We said that I and J are conflict if they are operations by different transaction on the same data item, and **at least one of these instructions is a write operation**

Our DBMS nowadays follow this, but when it do $\rightarrow$ it doesn't swap operations $\rightarrow$ it must have some algorithm to guarantee conflict serializable

Conflict serializable is an accepted definition when we said we want correctness (not the best)



Example of a schedule that is not conflict serializable.

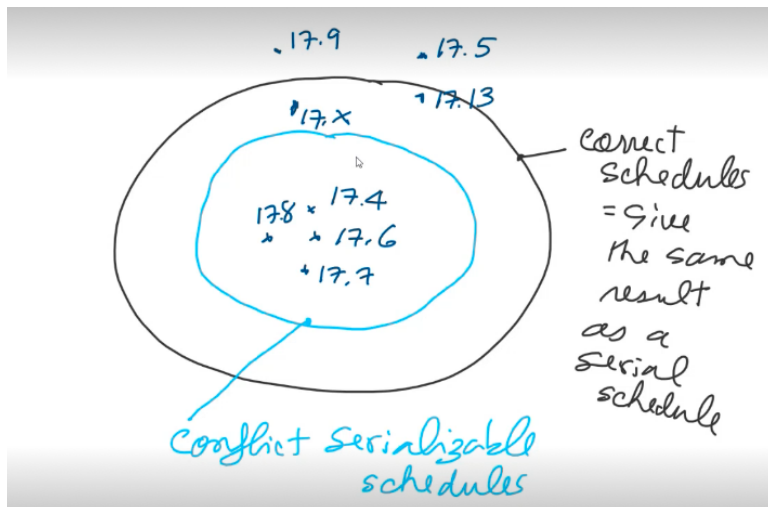


Conflict Serializability (Cont.)

- Example of a schedule that is not conflict serializable:

T_3	T_4
read (Q)	
write (Q)	write (Q)

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$.



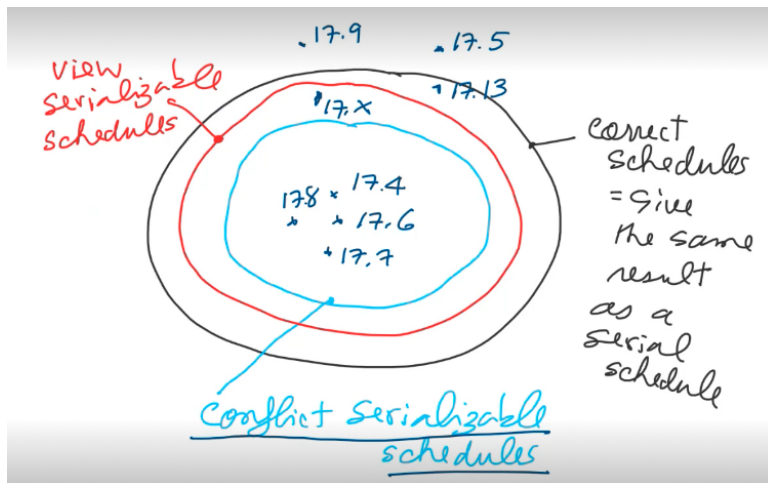
- incorrect schedule won't be allowed to run, 17.x and 17.13 will also not be allowed to run, if you set level of consistency to be serializable.
- Researchers try to move the blue circle as close to the black one as possible
- how to guarantee the black one?
 - need to be able to access everything, not just sql statement. (if 1 day, your scheduler has access to source code → achieved)
 - can't just determine from sql
- Now, we just use the blue one.

However, the scheduler not try to swap things like that, it will follow an algorithm that in turn guarantees conflict serializability (discussed in chapter 18)

there also another standard

✓ View Serializable Schedules

if it view equivalent to serial schedule



View Serializability

- Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q ,
 1. If in schedule S , transaction T_i reads the initial value of Q , then in schedule S' also transaction T_i must read the initial value of Q .
 2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j .
 3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S' .
- As can be seen, view equivalence is also based purely on **reads** and **writes** alone.
- if above 3 conditions are met, S and S' are view equivalent (view equivalent conditions)

- A schedule S is **view serializable** if it is view equivalent to a serial schedule.
- Every conflict serializable schedule is also view serializable.
- Below is a schedule which is view-serializable but *not* conflict serializable.

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

- What serial schedule is above equivalent to?
- Every view serializable schedule that is not conflict serializable has **blind writes**.

Arrange T27, 28, 29 → give the same result as running this?

- read and then write (update statement)
- read, then modify, then update
- T28, T29 → write Q without read? → blind writes
- view serializable schedule allows you to have blind write, but conflict serializable schedule don't allow

Normal write

Read x
write x

Update T1
Set $x = x + 1$
where
read

when you see read x write x, it kind of like you have to get x data (read) first, in order to update(write). Read old values before writing

- read and then write
- read first, write later
- refer to previous x

Blind write

Write x
blind write

Update T1
Set $x = 500$
where

- T28, 29 → write (Q) without pre read

Example

T_1	T_2
read (A) write (A)	
	read (A) write (A)
read (B) write (B)	
	read (B) write (B)

Schedule 3

T_1	T_2
read (A) write (A) read (B) write (B)	
	read (A) write (A) read (B) write (B)

Schedule 6

- Let check if these are view equivalent based on 3 conditions
 - rule 1
 - T1 read the initial value of A first in schedule 3, and also read A first in schedule 6 (checkkkk)
 - T1 read B first in S(the initial value)(schedule 3), T1 read B first in S'(schedule 6 too)
 - rule 2 (same source)
 - (Schedule 3)T2 read A, and this A produce by T1 in S
 - (schedule 6) T2 read A, and this A produce by T1 in S'
 - T2 read B produced by T1, in both S, and S'
 - rule 3
 - T2 write A last in S, T2 also write A last in S'
 - T2 write B last in S, T2 also write B last in S' too

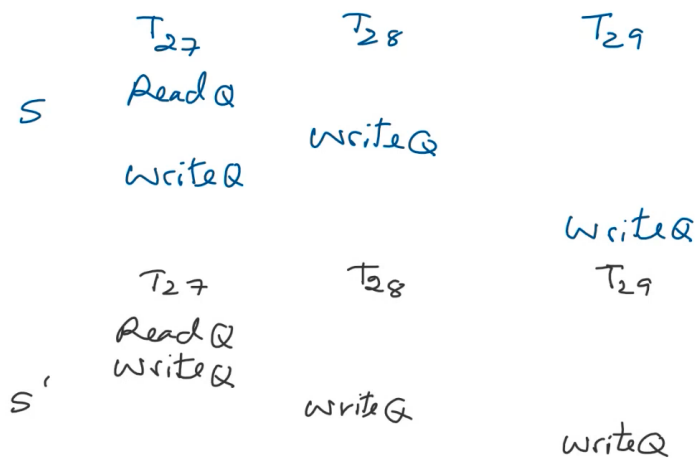
This 2 are view equivalent

and becasue scedule 6 is serial → schedule 3 is view serializabel schdule.

- A view serializable schedule s is a schedule which is view equivalent(3 conditions) to a serial schedule.
 - 3 conditions of view equivalent

Example (17.x)

Are S and S' view equivalent



Are they view equivalent and why?

- S' is serial schedule, so in fact, we checking is s a view serialize
- rule 1:
 - checkkkk, both read initial value of Q
- rule 2:
 - same source → no read data, only write (free)
- rule 3:
 - checkkkk, write final Q in both S and S'

In summary, conflict serializable is more practical, view serializability need certain condition to be check and it hard to check? and the benefit is simply → blind write

Every view serializable schedule that is not conflict serializable has blind writes

Most dbms nowadays refer to conflict serialize schedule not the view one

- conflict serializable → easier to implement



for levels of consistency, serializable either mean conflict serializable (more common) or view serializable

There important assumption whenw e consider conflict and view serialibility → which is all transaction succeed, all commit successfully, no failure. but in

reality, failure take place, we need to consider it

✓ Correctness level when we also consider failure

Important assumption when we consider conflict or view serializaiblity

- the assumption is → all transaction succeed → commit successfully no failure
- but in reality, failure take place, we have to consider failure.

🌱 Recoverable Schedule



Recoverable Schedules

Need to address the effect of transaction failures on concurrently running transactions.

- **Recoverable schedule** — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i appears before the commit operation of T_j .
- The following schedule (Schedule 11) is not recoverable

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

- actually, the sources should be commit before the transaction
- the source must commit first before the reader could commit
 - this case reader commit first before the source
- what if T8 eventually failed?
 - it mean T9 read value and commit but the source (T8) rolled back → T9 obtain wrong result (read from rolled back transaction)
 - we don't want this

This kind of schedule is called a **non-recoverable schedule** because T9 read A and commit, T8 rolled back, we can't rolled back T9. T9 obtain a wrong value

- What we want? - a recoverable schedules

Definition of Recoverable schedule - if a transaction Tj reads a data item previously written by a transaction Ti, then the commit operation of Ti appears before the commit operation of Tj

- in fact, recovery schedule is not enough (good condition but still not enough, we need **cascadeless schedule**)
- because we can have kind of recoverable schedule that lead to cascadesless roll back (example below)

Cascading Rollbacks

This slide show cascading rollback phenomenon (problem)

- **Cascading rollback** – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A)		
	read (A) write (A)	
abort		read (A)

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work

They read in sequence...

- Assume that this is a recoverable schedules → which mean T10 must commit before T11 and then; before T12 too
- eventually, T10 failed, what happend (abort, rolled back)
- because T10 rolled back, T11 and T12 must also be rolled back
 - waste money and time → undoing lot of work

- we don't really want this, we don't want this kind of cascading rollback

we want cascadeless schedules

- in cascadeless schedules → cascading rollback can't occur

(default in DBMS nowadays)(every cascadeless is recoverable schedule)



Cascadeless Schedules

- **Cascadeless schedules** — cascading rollbacks cannot occur;
 - For each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every Cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless

- the source must commit first before the reader can read.
- cascadeless must be recoverable
 - similar concept to recoverable
 - recoverable → source commit before reader commit
 - cascadeless → source commit first before reader could read
 - strict version of recoverable
- the value must commit before T_j could read

Standard for us for scheduling

What is good scheduling scheme?

A good scheduling scheme should guarantee

- conflict or view serializable schedules
- recoverable or cascadeless schedules

A good scheduling scheme should guarantee conflict or view serializable, this means when user, programmer submit transaction, the scheduler of DBMS should guarantee to have correct result

- and correct means conflict or view serializable schedules.

as mentioned earlier, view serializable is more generous but more complicated

- so in practice, we use **conflict serializable schedules as a standard**.

Also when we consider failure, we need to have recoverable or cascadeless schedules.



A good scheduling scheme should guarantee → conflict or view serializable schedules/ → recoverable or cascadeless schedules.

Who implement this/ ensure this good scheduling scheme? People or DBMS

- actually, it is possible for human programmer to do all this, but doesn't make sense. Should be DBMS, system software
- in the past, it was the job of human programmer. We all know that writing program to meet user requirement is hard enough, so all these things related to this topic should be handled by DBMS. → luckily, now, DBMS can handle this.
- we'll study later on how to work with this if one day we work without DBMS.
 - if one day we have to build scheduler of DBMS by ourselves.

Way to say what we want

- the correctness we want (way to say) → level of consistency (isolation level)
 - lower degree of consistency useful for gathering approximate information about the database. and therefore reduce the workload of

DBMS

- high level of correctness → more overhead of scheduling, DBMS work hard.

Levels of consistency (Isolation levels)



Levels of Consistency in SQL-92

- **Serializable** — default
- **Repeatable read** — only committed records to be read.
 - Repeated reads of same record must return same value.
 - However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- **Read committed** — only committed records can be read.
 - Successive reads of record may return different (but committed) values.
- **Read uncommitted** — even uncommitted records may be read.

- Different of level of consistency → the result of the same program may not be the same.
- serializable is a default, academically, but not practically
- if not set → use default (mostly, read committed)

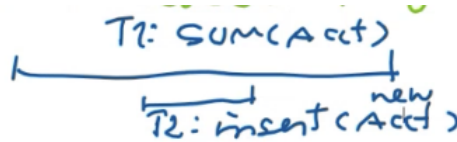
1) Serializable - conflict or view serializable schedules (garuntee by DBMS, there are several algorithm to garuntee this) + cascadeless schedule (the source must commit first before the reader can read)(casecadeless also recoverable) + garuntee no phantom phenamenon(insert will not get in)

1) Serializable: Conflict or view serializable schedules + cascadeless schedules + guarantee no phantom phenomenon. levels)

$T_1: \text{SUM}(\text{Acct})$
 $T_2: \text{insert}(\text{Acct})$ ^{new} — Insert of new account by T_2 should not be seen by T_1 but it is.

- cascadeless schedule also recoverable scedule

- what is phantom phenomenon?



- you have transaction T1 doing sum(ACCT)
 - t2 insert new account
- Insert of new account by T2 should not be seen by T1 but it is.
- the newly insert account and the sum won't conflict → different data item → **weak point of conflict serializability** (sum of T1 may include the newly inserted value → should not be a case, because it should be like T1 follow by T2) Insert of new account T2 should not be seen by T1 but it is.
 - conflict serializability not consider insertion
 - which should not be a case → because serializable is T1 first follow by T2
- Weak point of conflict serializable
- guarantee no phantom phenomenon, yayy (no insert could get in)
- 2) Repeatable read (allow insertion during read)(level lower than serializable)
 - only committed record to be read (cascadeless schedule)
 - when you want to read, you can't read uncommitted data → this guarantee cascadeless schedule(source must commit first before the reader can read)
 - Repeated reads of same record must return same value
 - it means this is conflict serializable → you don't see value write by someone younger.
 - However, transaction may not be serializable - it may find some records inserted by a transaction but not find others.
 - phantom phenomenon can still take place (it may find some record inserted)

- almost the same as serializable, but the phantom phenomenon may take place.

3) Read Committed

- garuntee cascadeless schedule
- only committed records can be read.
- Successive reads of record may return different (but committed) value.
- (concentrate on read only commit record, not garuntee conflict or view serializability.) - casecadeless schedule

4) Read uncommitted (Dirty read)

- even uncommitted record may be read.
- casecadeless schedules are not garunteed.

all this don't allow dirty write → overwrite value that update but haven't commit yet

Different setting give different result



Levels of Consistency

- Lower degrees of consistency useful for gathering approximate information about the database
- Warning: some database systems do not ensure serializable schedules by default
- E.g., Oracle (and PostgreSQL prior to version 9) by default support a level of consistency called snapshot isolation (not part of the SQL standard)
- Most vendor set default to be read-committed, because they know customer only concern about the response time, they don't care about correctness
 - nowadays, some use level of consistency for performance tuning.
- if want nice performance, relax correctness.

Nowadays, just tell DBMS you want what level of consistency. (don't have to use any technique)

Setting



Transaction Definition in SQL

- In SQL, a transaction begins implicitly.
- A transaction in SQL ends by:
 - **Commit work** commits current transaction and begins a new one.
 - **Rollback work** causes current transaction to abort.
- In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
 - Implicit commit can be turned off by a database directive
 - E.g., in JDBC -- `connection.setAutoCommit(false);`
- Isolation level can be set at database level
- Isolation level can be changed at start of transaction
 - E.g. In SQL **set transaction isolation level serializable**
 - E.g. in JDBC -- `connection.setTransactionIsolation(Connection.TRANSACTION_SERIALIZABLE)`

✓ Problem that prevent correctness (concurrency control based on problem concept)

(if we can solve problem, we can get correctness)

The 4 concurrency control problem

- they are the problem that prevent correct schedule.
- if they are solved → we get correct schedules

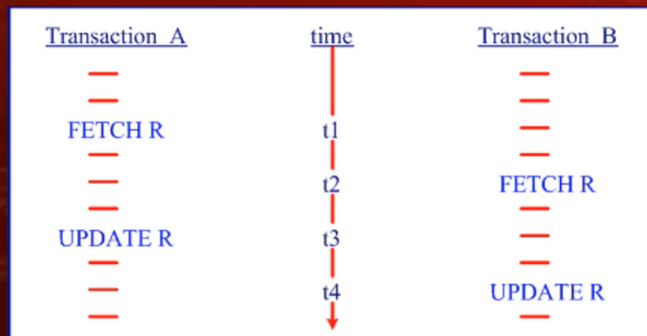
Originally, they were 3 of them

1. The lost update problem
2. The uncommitted dependency problem
3. The inconsistent analysis problem
4. (now we have) phantom phenomenon

1. The lost update problem (not conflict serializable)

- you lost value as written by A, B overwrite it → work done by A is lost.

1. The *lost update* problem,
2. The *uncommitted dependency* problem, and
3. The *inconsistent analysis* problem.



Transaction A loses an update at time t4

- is this conflict serializable? no, because you can't transform this into serial schedule (fetch r and update r conflict)
 - if you set your system to be serializable, this one won't come out.
- B read old value of R, it suppose to read after update R in transaction A.
 - it like running B alone, the work that done by A is lose.

If we use X lock, both X lock R, read R, unlock R → still wrong, doesn't help

2. The uncommitted dependency problem



Transaction A becomes dependent on an uncommitted change at time t2

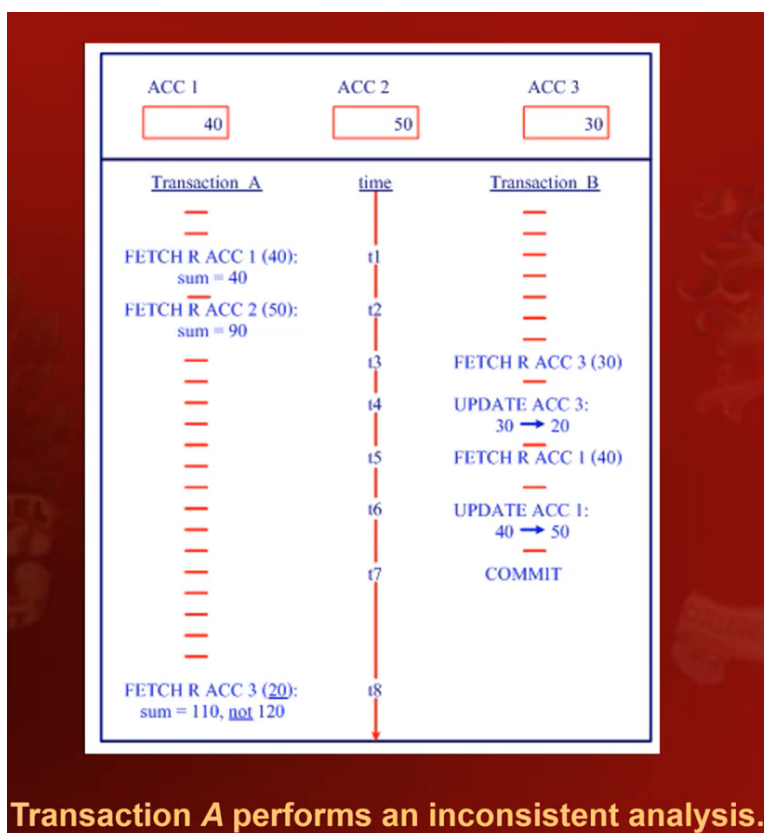


Transaction A updates an uncommitted change at time t2, and loses that update at time t3.

- First
 - not cascadeless schedule
 - Dirty read
- Second
 - Dirty write

3. The inconsistent Analysis (not conflict serializable, and incorrect result)

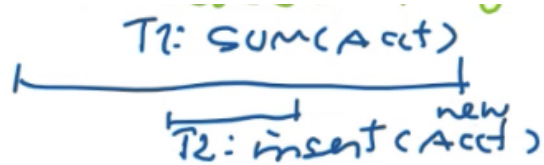
- read only transaction compute read from the data, and some other transaction update the data
 - phantom phenomena is the special case of inconsistent analysis



- is this conflict serializable schedule? no, because you have read(r, acc3) and update (r, acc3) → can't swap (conflict)
- does it give correct result? no

(4) Phantom phenomenon

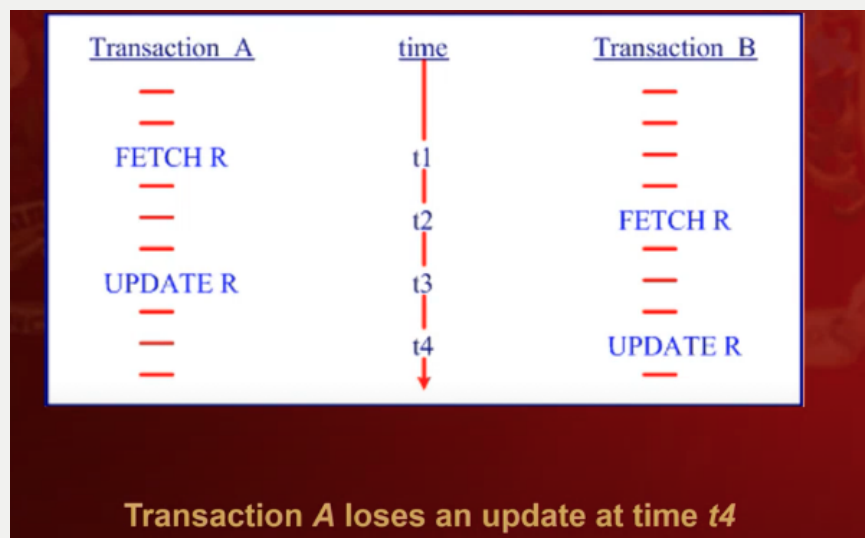
- you have T1 and T2 insert new account



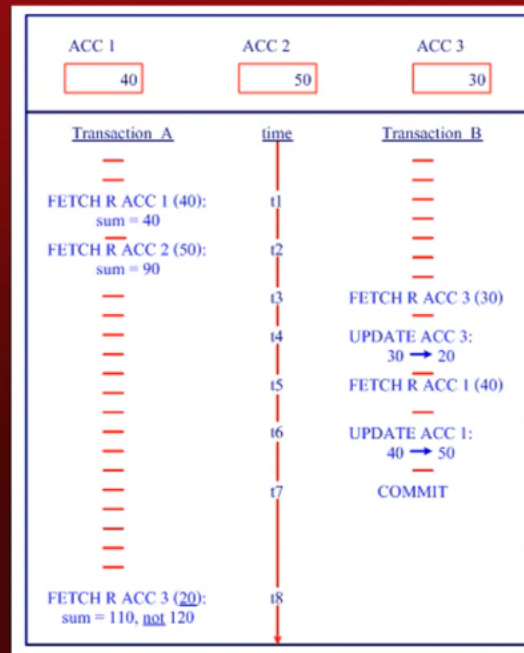
- when we talk about conflict serializable, write(A) write(b) not conflict → work on different data item.
 - newly insert account and the sum won't conflict and could get in

👉 Solution

- lot of people don't care about this topic here → because they said they have locking
 - in database, we don't lock process, we lock resources
 - like if you working on row A, you want to read row A, you can lock row A using read-lock → some body else can read the same row, but other process can't update row A. when finish read → unlock
 - X lock → lock for update
1. lost update problem → Lock doesn't help



- Lock R → finish reading unlock, *2
 - X Lock R → update unlock * 2
 - not help
2. uncommitted dependency problem → lock doesn't help
3. inconsistent analysis → lock doesn't help



4. not help too

By just locking doesn't help

Why can't locking solve 4 concurrency problem?

these problem are like the problem over period of time, but lock is a point of time (only 1 item) → lock is useful for point of time, like withdraw money → So locking alone doesn't help (it need to be locking + something)

Back to levels consistency

1) Serializable

- *prob1 -4 are all solved*

2) *Repeatable rad*

- *prob 1 - 3 are solved (not 4)*

3) Read committed

- *prob 2 is solved*

4) Read uncommitted

- *even prob2 is not solved.*

For programmer, you can just set this → don't really need to care about how it actually work, but let look at how it implement → chapter 18
